SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Preksha Prakash	**Reg. No.:**	1922260048
Date:	7-10-2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

1. Problem Understanding

Context & Industry Need

In modern cloud-native architectures, enterprise applications must maintain high availability (99.99%+ uptime) while rapidly evolving their software features. A key bottleneck in continuous delivery is the relational and non-relational database schema evolution. Traditional approaches rely on scheduled maintenance windows or locking tables, leading to costly application downtime, broken client requests, or catastrophic data corruption during failed deployments.

Problem Statement

Designing a Zero-Downtime Database Migration and Schema Evolution Platform that automates schema changes, ensures backward compatibility across co-existing application versions, detects runtime and static compatibility risks, coordinates automated deployment and rollback strategies, and seamlessly integrates into modern CI/CD pipelines.

Core Objectives & Scope

- Static & Dynamic Schema Validation: Validate SQL/NoSQL DDL scripts before execution to catch non-backward-compatible operations (e.g., dropping columns, changing data types, synchronous locking).
- Backward-Compatible Migration Strategy Support: Enforce multi-phase migrations (Expand-Contract / Parallel-Run pattern).
- Multi-Version Application Coordination: Enable multiple versions of application microservices (Version N and Version $N+1$) to operate concurrently against the same database schema during rolling deployments.
- Risk Detection & Compatibility Checks: Detect lock risks, large table migration impacts, and schema drift.
- Automated Rollback & Recovery: Maintain historical state and execution triggers to safely revert schema changes without data loss.
- CI/CD Integration & Observability: Integrate into pipelines (GitHub Actions, GitLab CI, Jenkins) and provide real-time database health metrics during migration.

2. Stakeholder & Challenge Analysis

Key Stakeholders & Roles

1. DevOps / SRE Engineers: Responsible for deployment reliability, zero-downtime executions, system availability, and monitoring pipeline success.
2. Database Administrators (DBAs): Responsible for database health, query optimization, lock monitoring, data integrity, and compliance.
3. Software Developers: Responsible for writing feature-level migration scripts, maintaining backward compatibility, and managing application model updates.
4. Product Owners & Business Stakeholders: Concerned with system reliability, uninterrupted user access, and fast time-to-market for new software capabilities.

Key Technical Challenges

- Exclusive Table Locking: DDL operations (such as `ALTER TABLE`, `ADD COLUMN DEFAULT`, or index creation) can acquire exclusive table locks, blocking live traffic.
- Schema-Application Coupling: Application instances running version N will crash if version $N+1$ removes or alters database columns expected by version N.
- Data Consistency During In-Flight Migrations: Backfilling historical data while live writes occur requires asynchronous triggers or dual-writing strategies.
- Rollback Complexity: Undoing a destructive database migration (like a dropped column) is inherently non-trivial once new data has been written in production.

3. Current System Analysis

Existing Systems / Traditional Approaches

Most current organizations rely on manual SQL execution scripts, basic CLI migration tools (e.g., basic Liquibase, Flyway CLI, or framework-native tools like ORM migrations), or off-peak maintenance windows.

Strengths of Current Approach

- Straightforward setup for early-stage development.
- Easy tracking of sequential version numbers (e.g., `V1__init.sql`, `V2__add_user_col.sql`).
- Clear developer familiarity with basic migration files.

Weaknesses & Critical Gaps

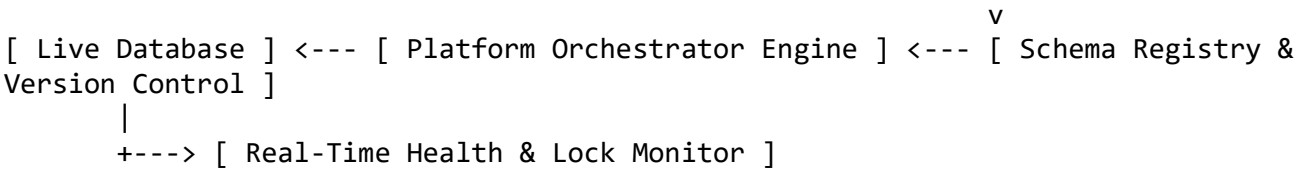
- Lack of Safety Checks: Standard migration runners do not parse SQL to warn against blocking DDL operations (e.g., non-null column additions without defaults).
- Monolithic Lock Execution: Migrations execute synchronously during deployment, causing requests to time out.
- All-or-Nothing Rollbacks: If a deployment fails halfway through, rolling back database schemas can leave the production data corrupted or inconsistent.
- Zero Coexistence Support: Modern blue-green or canary deployment models fail if the schema is not decoupled from the deployment lifecycle.

4. Proposed Software Solution & Software Engineering Principles

Architectural Solution

The proposed solution is a microservice-based Zero-Downtime Schema Evolution Platform built on the Expand-Contract (Parallel Change) Design Pattern.

```
[ Developer PR ] ---> [ CI/CD Pipeline ] ---> [ Migration Linter & Risk Engine ]
                                                          |
```



Key Engineering Modules & Features

A. Static Schema Linter & Safety Validator

- Uses abstract syntax tree (AST) parsers to analyze incoming migration scripts before execution.
- Enforces rules forbidding unsafe DDL operations (e.g., flagging `DROP COLUMN` and requiring a 2-phase deprecation workflow instead).

B. Expand-Contract Orchestrator

Automates multi-phase schema updates across deployments:

1. Expand Phase: Add new columns, tables, or non-blocking triggers. Schema remains fully backward-compatible with Version N.
2. Transition Phase (Dual-Write / Backfill): Application updates asynchronously backfill data. Both Version N and $N+1$ run concurrently.
3. Contract Phase: Safe removal of obsolete columns or triggers after all clients upgrade to Version $N+1$.

C. Real-Time Database Health & Lock Guard

- Continuously checks lock escalation, query latency, replication lag, and connection pools during migration runs.
- Implements automatic circuit breakers: aborts migration and releases locks if database CPU exceeds 80% or query latency spikes beyond baseline.

Core Software Engineering Principles Applied

- Modularity: Separate modules for parsing/linting, execution orchestration, monitoring, and pipeline integration.
- Scalability & Isolation: Decoupled architecture using worker queues (e.g., Celery/RabbitMQ) to manage background backfills without degrading API endpoints.
- Security & Auditability: Fine-grained Role-Based Access Control (RBAC) ensuring DB credentials used by the platform follow the principle of least privilege, with complete audit logs of every executed statement.
- Maintainability: Immutable schema versioning using a centralized Schema Registry.

5. Methodology & Framework Justification (10% Weightage)

Software Development Life Cycle (SDLC) Model

- Agile / DevSecOps Lifecycle: Adopted to enable iterative releases and continuous integration. Schema migrations are treated as first-class code artifacts subject to code reviews, automated testing, and automated security scans.

Technology Stack Justification

- Orchestration & Backend: Go (Golang) / Python — chosen for lightweight concurrent execution, fast performance, and strong AST parsing libraries for SQL dialect analysis.
- Database Compatibility: PostgreSQL / MySQL / Distributed SQL — utilizing native online DDL algorithms (e.g., `pg_repack`, `gh-ost`, `pt-online-schema-change`) to prevent table locking during migrations.
- CI/CD Integration: Custom plugins / GitHub Actions / GitLab Runners exposing CLI and REST API endpoints.
- Observability: Prometheus & Grafana integrations for tracking migration runtime, database load, and active transaction locks.

Justification Summary

This methodology guarantees that schema changes are validated statically before hitting production, executed dynamically without exclusive table locks, monitored continuously with automated health checks, and fully reversible in the event of pipeline failures.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system,	Good analysis with most strengths and	Limited analysis with partial	Proposed solution is unclear or inappropriate;
	strengths, weaknesses, and identifies all gaps.	weaknesses identified.	identification of issues.	requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25